1. **¿Cuál es la diferencia entre una lista y una tupla en Python?**

La principal diferencia entre una lista y una tupla en Python es que las listas son mutables (se pueden modificar después de su creación), mientras que las tuplas son inmutables (no se pueden modificar).

Además, se diferencian en su sintaxis: las listas se definen con corchetes [] y las tuplas con paréntesis ().

- una_lista = ['Alba', 'Mikel', 'Jon']
- una_tupla = ('Alba', 'Mikel', 'Jon')

Debido a la inmutabilidad de las tuplas, estas suelen ser más eficientes y se utilizan cuando no vamos a querer que los datos cambien. Si en un momento dado queremos añadir o modificar algún elemento de la tupla, realmente lo que haremos será crear una nueva dupla. Por ejemplo:

una_tupla = ('Alba', 'Mikel', 'Jon') + ('Leire',)

Esta operación no está modificando la tupla sino que está creando una nueva que se almacenará en otra dirección de la memoria.

2. **¿Cuál es el orden de las operaciones?**

El orden de las operaciones es el mismo que en la jerarquía matemática: paréntesis (), exponentes **, multiplicaciones *, división /, suma + y resta -. (P E M D A S)

Hay que mencionar que cuando nos encontramos con operaciones del mismo nivel el orden a seguir es de izquierda a derecha. Las operaciones que son del mismo nivel se resuelven de izquierda a derecha. Estos serían, por un lado, los casos de las multiplicaciones y divisiones y, por otro, de las sumas y las restas. A modo de excepción, cuando tenemos varias operaciones con exponentes estás se resuelven de derecha a izquierda.

3. **¿Qué es un diccionario Python?**

Es un tipo de dato integrado que almacena colecciones de datos donde cada uno tiene una llave (key) y otro un valor (value). Estos se escriben mediante llaves {} y van separadas por comas. Por ejemplo:

```
alumno1 = {
        "Nombre" : "Jon",
        "Apellido" : "García",
        "Curso" : "5B"
}
```

En este ejemplo, nombre sería la llave y Jon sería el valor. Las características principales de un diccionario son las siguientes:

- Son dinámicos, es decir, pueden añadirse, modificarse o eliminarse elementos.
- Son indexados, es decir, se puede acceder a los elementos del diccionario a través de la llave.

- Son anidados, es decir, se pueden añadir diccionarios dentro de un diccionario.

Además, también se pueden incluir listas dentro de un diccionario.

Para acceder un elementos tenemos varias opciones, la primera sería con []:

```
print(alumno1['Nombre'])
```

o con la función get():

```
print(alumno1.get('Nombre))
```

Para modificar un elemento se puede hacer accediendo a él directamente:

```
Alumno1['Nombre'] = "Juan"
```

Si no existiera dicho elemento se añadiría automáticamente a nuestro diccionario.

A continuación, voy a explicar algunos métodos que son relevantes:

- items() nos devuelve una vista con los keys y los values del diccionario. Esta vista se puede convertir en una lista con la función list().
- keys() nos devuelve una vista con todas las keys del diccionario.
- values() nos devuelve una vista con todos los valores del diccionario.
- pop() busca y elimina la key que le pasamos como parámetro. Si le pasamos una key que no existe nos daría un error. Además, le podemos pasar como parámetro un mensaje para que nos devuelva si no encuentra el key.

4. ¿Cuál es la diferencia entre el método ordenado y la función de ordenación?

La primera es que .sort() es un método que solo existe en las listas y la función sorted() es una función integrada que existe con independencia de las listas y funciona para otro tipos de datos.

Además, .sort() no devuelve ningún valor, sino que modifica el objeto. Mientras que la función sorted() devuelve una nueva lista ordenada. Además, no modifica el objeto original.

5. ¿Qué es un operador de reasignación?

Es un operador que actualiza el valor de una variable de una forma más concisa. Se compone de un operador (+, -, *…) y el signo igual. Por ejemplo:

manzanas += 3 → De esta forma, habremos sumado más 3 manzanas a la variable de manzanas.

Cuando Python lee esta estructura hace lo siguiente, lee la variable la suma 3 y guarda el nuevo valor en esa misma variable. Por eso se llama operador de reasignación ya que se reasigna a sí misma el nuevo valor.